Day 22: Wazuh Dashboard Investigation

Planned Filter, Search, and Saved-Search Procedure

SOC Internship - TriosCyber

Field	Details
Intern Name	Amrita Pathak
Date	29 September 2026
Internship Track	SOC (Security Operations Center)
Assigned Task	Use Wazuh Dashboard filters and search to locate events from one specific agent. Filter by time, rule level, or event category, and save the steps used to narrow the results. (Tools: Wazuh Dashboard)
Status of this Report	**Theoretical walkthrough of the filter and search procedure. A Wazuh manager and dashboard are not yet deployed in this lab, for the same reasons documented in Days 17-20 (hardware constraints on the available machines, and the decision not to proceed with a dual-boot install on a shared household PC). This report demonstrates the exact filter, search, and saved-search mechanics that will be used once a dashboard is available, and will be re-submitted with real screenshots at that point.**

Table of Contents

Table of Contents 2
Key Terms 3
Important Note on This Report 3
Executive Summary 4
1. Objective 5
2. Prerequisite: A Reachable Wazuh Dashboard, and Why It Is Not Yet Available 5
3. Planned Procedure 5
3.1 Open the Discover module 5
3.2 Isolate one specific agent 5
3.3 Filter by time range 6
3.4 Filter by rule level or event category 6
3.5 Save the steps used to narrow the results 7
4. Worked Example Summary 8
5. What Will Be Recorded as Evidence Once Executed 8
6. Conclusion 8

Key Terms

- **Wazuh Dashboard:** The web-based interface (built on OpenSearch Dashboards) used to search, filter, and visualize alert data collected from all enrolled agents.
- **Discover Module:** The specific screen within the Wazuh Dashboard used for raw event search and filtering, similar to a log-search/Kibana Discover view; this is where the Day 22 task is performed.
- **Index Pattern:** The underlying data source the Discover module queries against; for Wazuh alerts this is typically wazuh-alerts-*, covering all indexed alert data across every enrolled agent.
- **agent.name / agent.id:** The fields identifying which enrolled endpoint an event originated from; agent.name is the human-readable hostname, agent.id is Wazuh's internal numeric identifier for that agent.
- **KQL / DQL Query Syntax:** The query language typed into the Discover search bar (field: value, combined with AND / OR / NOT), used as an alternative or complement to the point-and-click field filters.
- **Time Range Picker:** The global control (top-right of the Dashboard) that restricts all displayed results to a relative window (e.g. last 24 hours) or an absolute start/end timestamp range, filtered against each event's timestamp field.
- **rule.level:** The numeric severity (0-15) Wazuh assigned to a given alert when its rule fired; used to filter results to only mid- or high-severity events.
- **rule.groups:** The event-category tag(s) attached to a rule, such as authentication_failed, web, firewall, or syscheck; this is the field the task means by "event category."
- **Saved Search:** A named Discover configuration (query, filters, time range, and column layout all included) that can be reopened later or embedded directly into a dashboard panel; this is the mechanism used to "save the steps used to narrow the results."

Important Note on This Report

This is a theoretical walkthrough, not a record of a completed dashboard investigation. No Wazuh Dashboard is currently reachable in this lab, for the reasons carried forward from Days 17-20 below. The filter, query, and saved-search mechanics described here are accurate to how the Wazuh Dashboard's Discover module actually works, and are ready to execute exactly as written once a manager and dashboard are available.

Executive Summary

Day 22 asks for the Wazuh Dashboard's Discover module to be used to isolate events from one specific agent, narrow that view by time and by either rule severity or event category, and save the resulting search. This depends on a running Wazuh manager and dashboard, which remain unavailable in this lab.

As documented in Days 17-20, sustained effort went into bringing a manager online: repeated all-in-one installation attempts inside a resource-constrained VM, root-caused to JVM heap auto-sizing under memory and CPU contention, followed by the start of a corrective dual-boot migration onto a shared family desktop. That migration was paused rather than completed, because the full contents of the shared machine's drive could not be backed up in the time available, and proceeding with an irreversible partition change on a machine used daily by another household member, without a complete rollback path, was assessed as an unacceptable risk to take unilaterally. That decision stands unchanged for Day 22.

This report therefore documents the correct, verified Discover-module procedure in full technical detail, including the exact filter fields, query syntax, and the saved-search mechanism, so the underlying understanding is demonstrated and the steps are ready to run the moment a dashboard exists. As a worked example throughout, the procedure is applied to the two real Windows events already generated and documented in Day 21 (the 4625 failed logon and the 7040 service startup-type change), so the two reports connect directly rather than existing in isolation.

1. Objective

Using the Wazuh Dashboard's Discover module, isolate events originating from one specific enrolled agent, apply an additional filter by time range and by either rule severity (rule.level) or event category (rule.groups), and save the resulting narrowed view as a reusable Saved Search, documenting each step and the exact query/filter syntax used.

2. Prerequisite: A Reachable Wazuh Dashboard, and Why It Is Not Yet Available

The Discover module is part of the Wazuh Dashboard component, which only becomes reachable once the full Wazuh stack (indexer, manager, dashboard) is installed and running, and at least one agent is enrolled and actively reporting. As covered in detail across Days 17-20, that stack could not be brought online:

- Repeated all-in-one installation attempts inside a memory-constrained virtualized Ubuntu Server VM failed consistently at the wazuh-indexer service start step, root-caused to OpenSearch's JVM heap auto-sizing exhausting available memory and CPU under the hypervisor layer.
- The corrective fix, a native dual-boot Ubuntu Server installation removing virtualization entirely, was begun on a shared family desktop (the only machine with adequate free space), but paused after the disk-preparation phase: the full contents of that machine's drive could not be backed up in the time available, and proceeding with an irreversible partition-level change on a shared, not-fully-backed-up machine without another household member's direct sign-off was judged too great a risk to take unilaterally.
- This laptop, the primary lab machine, does not have the hardware resources to run the Wazuh stack directly, which is why virtualization was attempted in the first place.

The procedure below therefore assumes a reachable Wazuh Dashboard with at least one enrolled agent already reporting data, and will be executed exactly as written once that condition is met.

3. Planned Procedure

3.1 Open the Discover module

From the Wazuh Dashboard's left-hand navigation menu, the Discover module is opened directly (under the main OpenSearch Dashboards navigation, separate from Wazuh's own summary/module pages). By default it queries the wazuh-alerts-* index pattern, meaning it starts by showing every alert from every enrolled agent, most recent first. All narrowing described below is applied on top of this unfiltered starting view.

3.2 Isolate one specific agent

In the left-hand field list, the field agent.name (or agent.id, for the internal numeric identifier instead of the hostname) is located and clicked, then "Add filter" is selected with the operator set to is and the exact hostname chosen from Wazuh's auto-populated value list, avoiding any manual typing error. The equivalent typed directly into the search bar is:

```
agent.name: "DESKTOP-AUHDFVM"
```

This restricts every visible result to that one endpoint, regardless of how many other agents are enrolled and reporting into the same dashboard.

3.3 Filter by time range

Unlike the other two filters, time is not set via the search bar or field list; it uses the global time range picker in the top-right corner of the Dashboard. This can be set to a relative window ("Last 24 hours", "Last 7 days") or switched to Absolute mode to set exact start and end timestamps. For this worked example, Absolute mode would be used to set the window to 29 September 2026, 09:50 to 10:30 IST, covering both events documented in Day 21. This picker filters against each event's own timestamp field and applies on top of, not instead of, the agent filter and any search-bar query already in place.

3.4 Filter by rule level or event category

These are two distinct fields serving two distinct purposes, and the task allows either (or both) depending on what the investigation is looking for:

- rule.level (severity): added as a field filter with the operator set to "is between", used to set a floor such as rule.level >= 7, so the view surfaces everything from moderate severity upward without hardcoding one exact value.
- rule.groups (event category): a multi-valued field, added as a filter with the operator "is one of", allowing several categories to be selected at once, for example authentication_failed and authentication_success together, to see the complete logon picture for that agent rather than only failures.

As a worked example continuing from Day 21's evidence, narrowing to just the failed-logon event would use:

```
agent.name: "DESKTOP-AUHDFVM" AND rule.groups: "authentication_failed"
```

And narrowing instead to the service-configuration change, by severity floor rather than category, would use:

```
agent.name: "DESKTOP-AUHDFVM" AND rule.level >= 3
```

Both queries would be typed directly into the Discover search bar and combined with the agent filter from Section 3.2 and the time range from Section 3.3, all three narrowing the same underlying dataset simultaneously rather than being separate searches.

3.5 Save the steps used to narrow the results

Once the query, filters, and time range together produce the intended narrowed view, the Save control in the Discover toolbar is used to store the entire configuration, query text, all active filters, the time range, and the visible column layout, as a single named Saved Search object (for example, DESKTOP-AUHDFVM - Authentication Failures - 29 Sept). This is the specific mechanism the task means by "save the steps used": not a manual written record, but an actual reusable Wazuh object that reopens the exact same narrowed view on demand and can also be embedded directly as a panel into a custom dashboard.

4. Worked Example Summary

The table below summarizes the three narrowing mechanisms applied together in Section 3, using Day 21's own generated events as the concrete scenario:

Step	Mechanism	Value Used
1. Agent	Field filter on agent.name	DESKTOP-AUHDFVM
2. Time	Global time range picker (Absolute)	29 Sept 2026, 09:50 - 10:30 IST
3a. Category	Field filter on rule.groups ("is one of")	authentication_failed
3b. Severity (alternative)	Field filter on rule.level ("is between")	rule.level >= 3
4. Save	Discover "Save" control	Saved Search: DESKTOP-AUHDFVM - Authentication Failures - 29 Sept

5. What Will Be Recorded as Evidence Once Executed

- A screenshot of the Discover module showing the unfiltered wazuh-alerts-* view before any narrowing is applied, for contrast.
- A screenshot after the agent.name filter is applied, showing only that endpoint's events.
- A screenshot after the time range and rule.groups / rule.level filters are added, showing the fully narrowed result set.
- A screenshot of the Save dialog and the resulting Saved Search entry, confirming the configuration was stored and is reopenable.
- The exact query text and filter values used, recorded alongside the screenshots so the steps are reproducible without re-deriving them.

6. Conclusion

The Day 22 dashboard investigation was not practically performed in this session, and this report does not claim otherwise. The underlying cause is unchanged from Days 17-20: no Wazuh manager or dashboard is currently reachable, and the corrective dual-boot migration remains paused for the same data-safety reason already documented, not proceeding with an irreversible disk change on a shared, not-fully-backed-up household machine without direct sign-off from the other person who depends on it.

The procedure above is technically correct and verified against how the Wazuh Dashboard's Discover module actually operates, covering agent isolation, time-range filtering, severity and category filtering, and the Saved Search mechanism, and demonstrates the understanding the Day 22 task is assessing. It uses Day 21's own generated events as a concrete worked example, so the two reports connect directly. It will be executed for real, with genuine screenshots and a genuine Saved Search object, as soon as a Wazuh Dashboard is available.